

TP3 - Intro à la statistique spatiale 2A - carto avec R - corrigé

OBJECTIFS DU TP :

- Le but de ce TP est de se sensibiliser à la discrétisation de variables, et découvrir des packages de cartographie avec R. Nous verrons notamment l'utilisation du package `maps` pour les cartes "statiques" et `leaflet` pour les cartes dynamiques. Vous trouverez les cheatsheets correspondantes [ici](#) et [ici](#).
- Afin d'utiliser une version de R plus récente (et une version des packages les plus récentes aussi), vous travaillerez sur le datalab (plateforme du sspcloud, service de l'Insee) : <https://datalab.sspcloud.fr>.

Les packages nécessaires pour ce TP sont listés ci-dessous :

```
# Chargement des packages
library(dplyr)
library(sf)
library(maps)
library(classInt)
library(leaflet)
library(openxlsx)
```

Exercice 1

Le but de cet exercice est de discrétiser une variable continue et d'en observer les différents résultats selon la méthode choisie.

Vous utiliserez le **fond des communes de France métropolitaine (fonds/commune_francemetro_2021.gpkg)** sur lequel vous calculerez une variable de densité. Pour la variable de population, vous disposez notamment du fichier **"Pop_legales_2019.xlsx"** présent dans le dossier **data**.

1. Commencez par vous créer votre jeu de données (jointure et création de variable). Attention avant de joindre vos données, il vous faudra d'abord homogénéiser la commune de Paris. Dans un fichier (fond communal), Paris est renseigné sous son code communal (75056). Dans l'autre, Paris est renseigné par arrondissement (75101 à 75120). Vous devrez donc **regrouper les arrondissements pour avoir une seule ligne pour Paris**. Cette ligne sera renseignée avec le CODGEO 75056.

```
# Import des donnees

# Fond communes France metropolitaine
communes_fm <- st_read("fonds/commune_francemetro_2021.gpkg") %>%
  select(code, libelle, surf)
```

```
Reading layer `commune_francemetro_2021' from data source
  `/home/onyxia/work/2A_stat_spatiale/2025-2026/TP3/fonds/commune_francemetro_2021.gpkg'
  using driver `GPKG'
Simple feature collection with 34836 features and 16 fields
Geometry type: MULTIPOLYGON
Dimension:      XY
Bounding box:   xmin: 99225.97 ymin: 6049647 xmax: 1242375 ymax: 7110480
Projected CRS:  RGF93 v1 / Lambert-93
```

```
# Import des population légales des communes en 2019
pop_com_2019 <- openxlsx::read.xlsx("data/Pop_legales_2019.xlsx")

# Correction pour la ville de Paris
```

```
pop_com_2019 <- pop_com_2019 %>%
  mutate(COM = if_else(substr(COM,1,3) == "751","75056",COM)) %>%
  group_by(code = COM) %>%
  summarise(pop = sum(PMUN19))

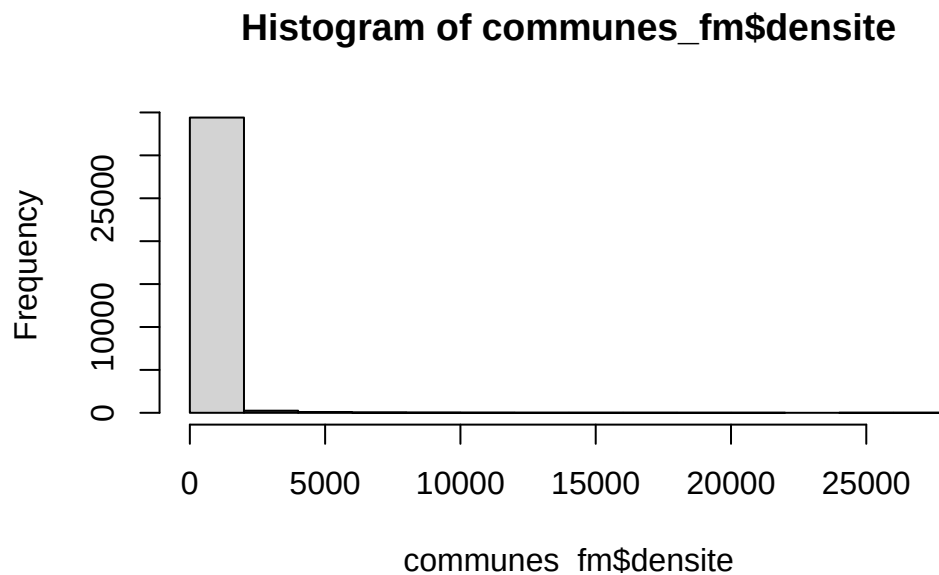
# Jointure
communes_fm <- communes_fm %>%
  left_join(pop_com_2019,
            by="code") %>%
  mutate(densite=pop/surf)
```

2. Regarder rapidement la distribution de la variable de densité.

```
summary(communes_fm$densite)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
0.00	18.30	40.34	163.25	95.72	27306.61

```
hist(communes_fm$densite)
```

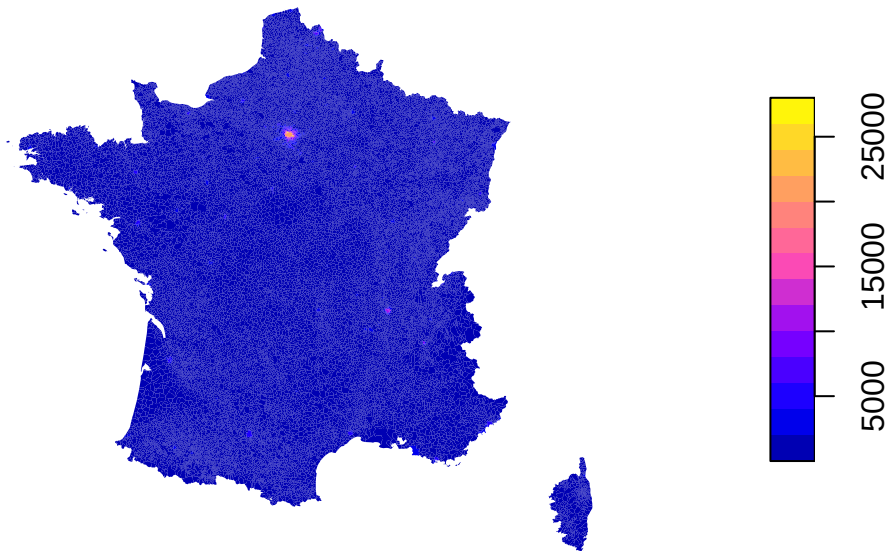


```
# Une très grande majorité de communes avec une faible densité
# Trois quarts des communes ont moins de 95 habitants au km2
```

3. On souhaite représenter la variable de densité sous forme d'une carte choroplèthe. Faire cela en utilisant la fonction `plot()`. La variable de densité sera sélectionnée par les crochets sur le modèle suivant : `ma_table["ma_variable_continue"]`. Ajouter également l'argument `border=FALSE` pour supprimer les bordures des polygones.

```
plot(communes_fm["densite"], border=FALSE)
```

densite



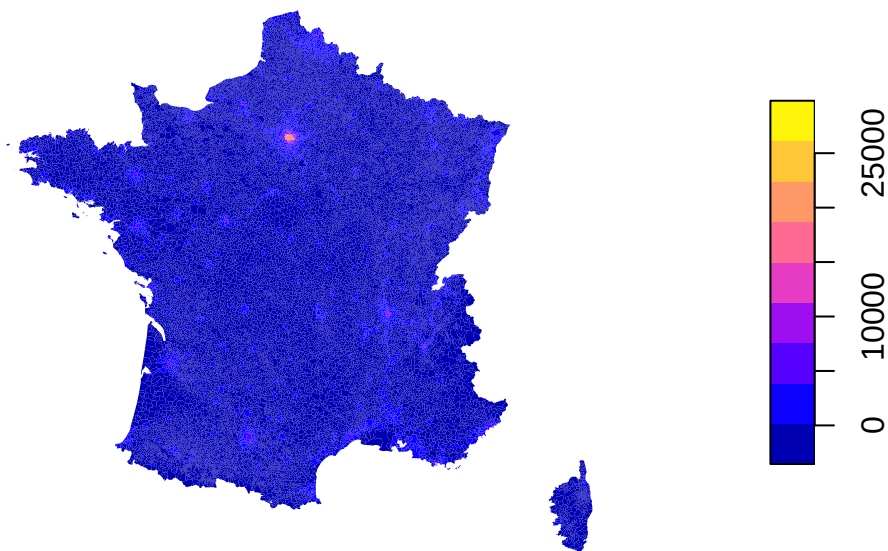
```
# Une représentation réalisée directement sur une variable continue peut être pertinente quand
# le nombre d'observations est très grand et quand la distribution est suffisamment équilibrée.

# Or, la densité de population des communes en France métropolitaine est une variable très
# déséquilibrée, avec une très grande majorité de communes en-dessous de 100 habitants/km2,
# une moyenne à 162 hab/km2 et quelques communes très denses.
# Ainsi, la carte construite est peu informative. (France très majoritairement rurale !)
```

4. On se rend compte que la carte est peu informative et qu'il vaut mieux discrétiser notre variable. Représenter le résultat de la discrétisation de la densité de population selon les méthodes des quantiles, jenks et des écarts-types, ainsi que la méthode `pretty`. Vous utiliserez la fonction `plot` en ajoutant l'argument `breaks=` + le nom de la méthode. Vous analyserez les différences entre les différentes cartes. Laquelle retiendriez-vous ? Pourquoi ?

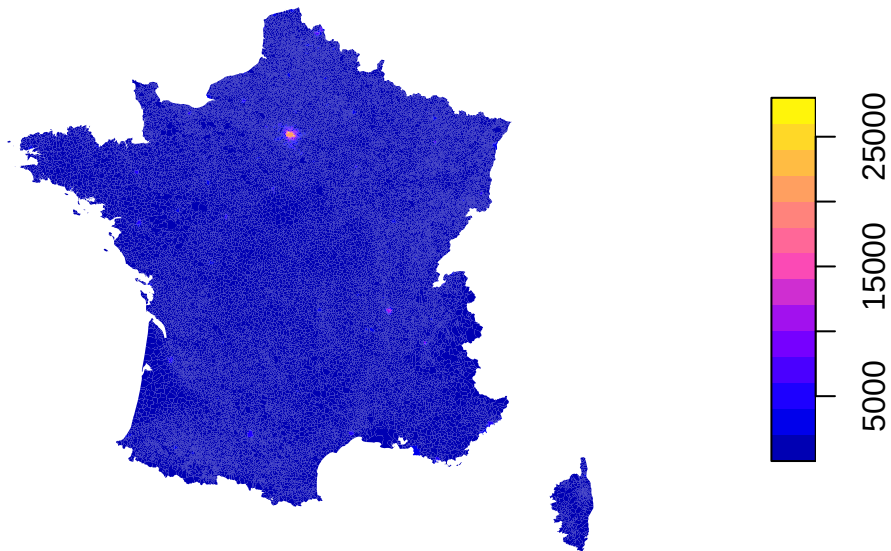
```
plot(communes_fm["densite"], breaks="sd", main="sd", border = FALSE)
```

sd



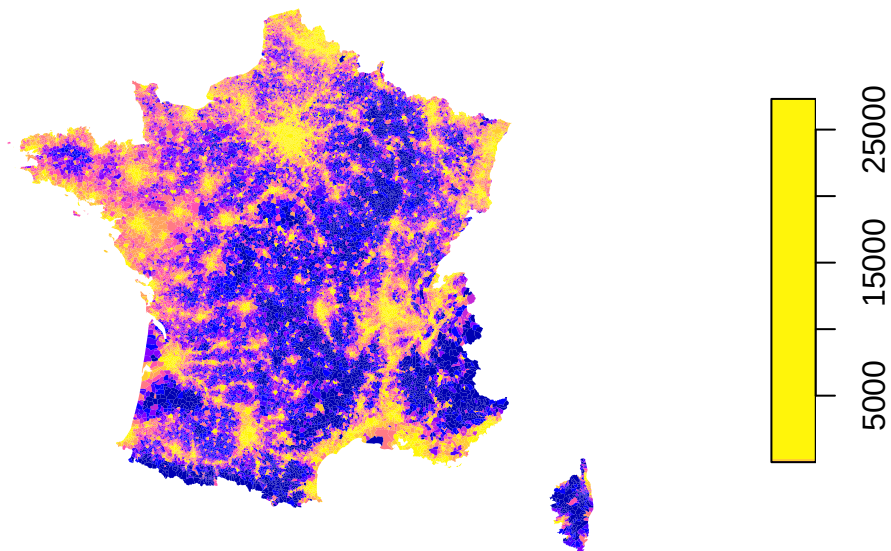
```
plot(communes_fm["densite"], breaks="pretty", main="pretty", border = FALSE)
```

pretty



```
plot(communes_fm["densite"], breaks="quantile", main="quantile", border = FALSE)
```

quantile

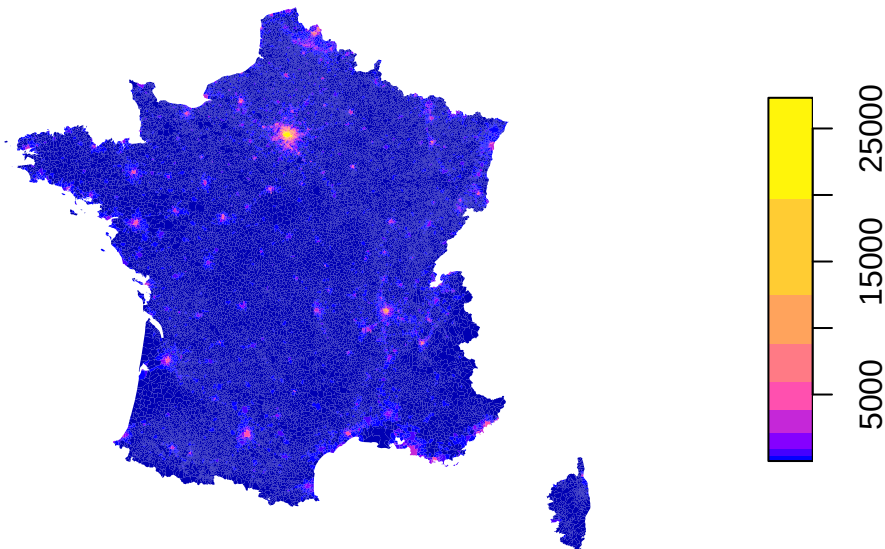


```
plot(communes_fm["densite"], breaks="jenks", main="jenks", border = FALSE)
```

Warning in classInt::classIntervals(v0, min(nbreaks, n.unq), breaks, warnSmallN = FALSE): N is large, and some styles will run very slowly; sampling imposed

Use "fisher" instead of "jenks" for larger data sets

jenks



```
# La méthode des écarts-types n'est pas adaptée aux données. La carte n'est pas très
# informative même si les grandes agglomérations commencent à apparaître.

# La méthode pretty est clairement inefficace pour fournir une carte informative.
# Cette méthode fournit des intervalles réguliers et arrondies (par exemple des classes
# de même amplitude par tranche de 2000).

# Les méthodes quantile et jenks semblent plus adaptées pour révéler la structuration du
# territoire entre agglomérations avec de fortes densités et les zones les plus éloignées
# de ces centres urbains. Néanmoins, on aimerait peut-être une classification intermédiaire
# entre ces deux méthodes. En effet, Jenks construit une classe de faible densité très
# nombreuse et la méthode des quantiles semble rassembler un peu trop de communes dans
# les classes les plus denses.
```

5. Pour obtenir une classification satisfaisante, il faut pouvoir comparer la distribution de la variable continue avec celle de la variable discrétisée. Le package `classInt` est très utile pour construire et analyser les classes.
 - a. Discrétiser la variable de densité avec la fonction `classInt::classIntervals()` avec la méthode des quantiles (argument `style`) et 5 classes (argument `n`). Vous pourrez vous appuyer sur le modèle ci-dessous. Analyser ensuite l'objet obtenu. Quelles informations contient-il ? Quelles sont les bornes des intervalles qui ont été construits ?

```
objet_decoupe <- classIntervals(
  ma_table$ma_var_continue,
  style = "quantile",
  n = 5
)
```

```
denspop_quant <- classIntervals(
  communes_fm$densite,
  style = "quantile",
  n = 5
)
str(denspop_quant)
```

```
List of 2
 $ var : num [1:34836] 1.52 504.92 41 12.08 5.66 ...
 $ brks: num [1:6] 0 15.3 29.5 55 121.1 ...
 - attr(*, "style")= chr "quantile"
 - attr(*, "nobs")= int 33739
 - attr(*, "call")= language classIntervals(var = communes_fm$densite, n = 5, style = "quantile")
```

```
- attr(*, "intervalClosure")= chr "left"
- attr(*, "class")= chr "classIntervals"
```

```
head(denspop_quant$var) # variable numérique de départ
```

```
[1] 1.522212 504.915254 41.001747 12.079616 5.662338 9.978960
```

```
denspop_quant$brks # bornes des intervalles construits
```

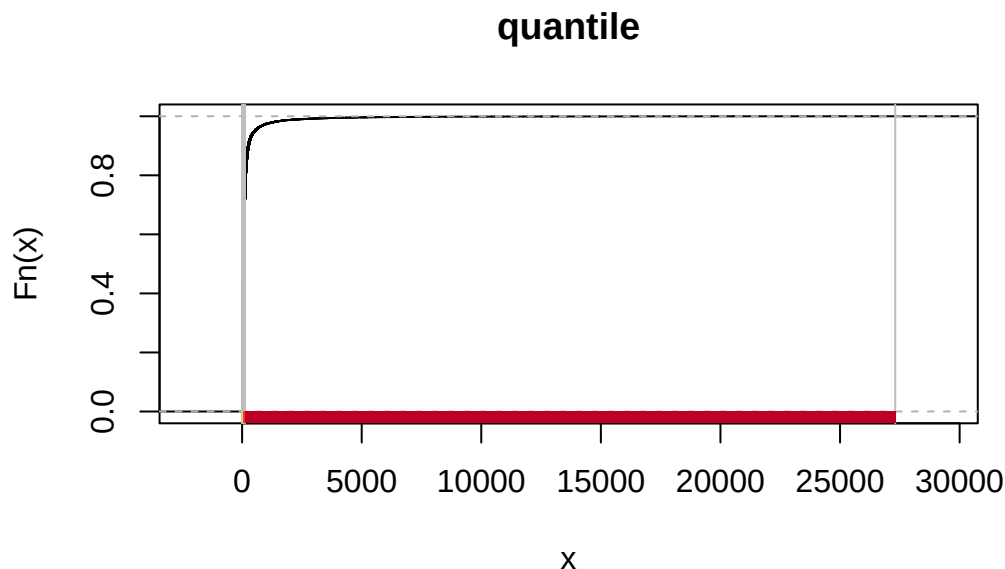
```
[1] 0.00000 15.34789 29.47814 55.04587 121.06509 27306.61157
```

```
# L'objet contient un sous-objet `var`, variable numérique de départ, et le sous-objet
# `brks` qui contient les bornes des intervalles construits. Ici `brks` a six valeurs,
# permettant de décrire les 5 intervalles demandés.
```

- b. Construire une palette de couleurs en lançant le code ci-dessous. Représenter ensuite l'objet précédent (découpage quantile) avec la fonction `plot()` et cette palette de couleur (argument `pal=`). Vous ajouterez l'argument `main =` à votre fonction `plot()` pour spécifier un titre. Analyser le graphique.

```
pal1 <- RColorBrewer::brewer.pal(n = 5, name = "YlOrRd")
```

```
plot(
  denspop_quant,
  pal = pal1,
  main = "quantile"
)
```



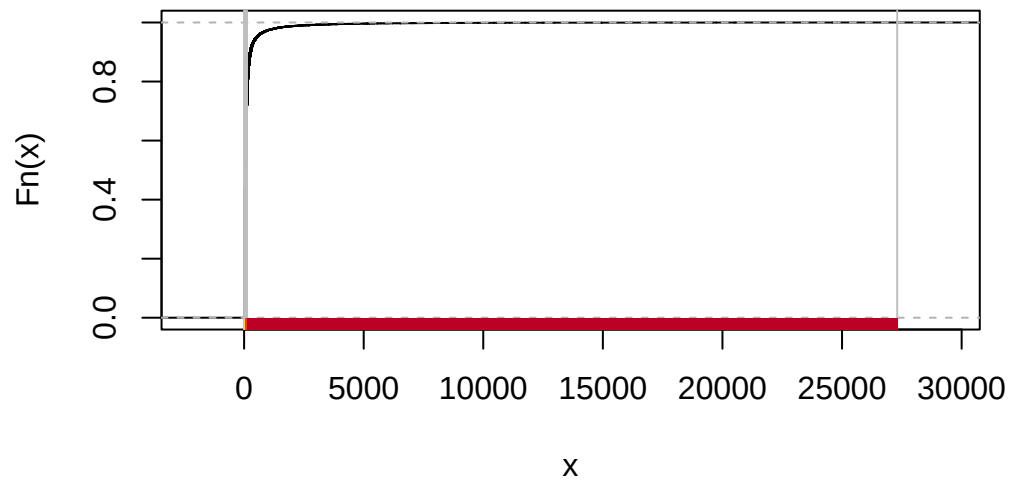
- c. Relancer l'analyse pour les méthodes `sd`, `jenks` et `pretty`.

```
analyser_discret <- function(method, nb_classes){
  denspop_c <- classIntervals(
    communes_fm$densite,
    style = method,
    n = nb_classes
  )
  print(denspop_c$brks)
  plot(
    denspop_c,
    pal = pal1,
    main = method
  )
  return(denspop_c)
}
```

```
# Avec 5 classes:
all_discret <- sapply(c("quantile", "sd","pretty","jenks"), analyser_discret, nb_classes = 5)
```

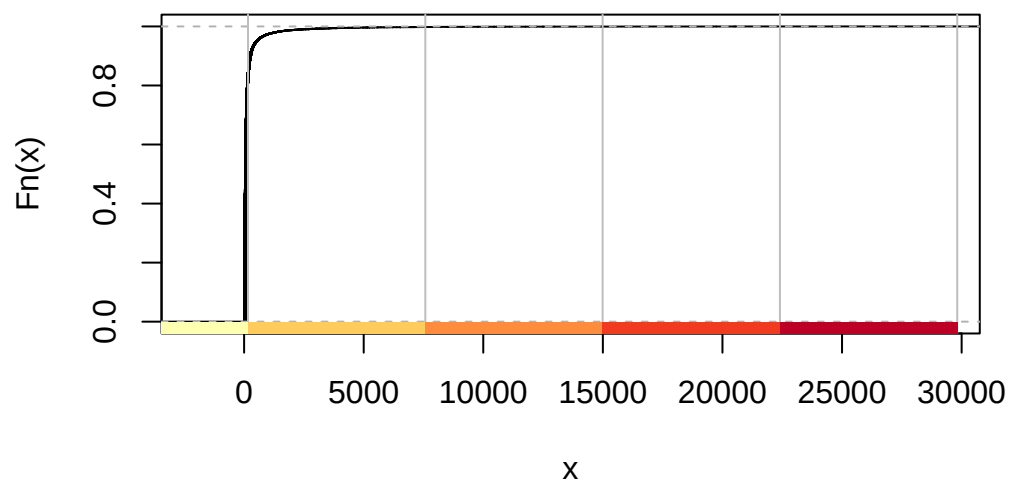
```
[1]      0.00000    15.34789    29.47814    55.04587   121.06509 27306.61157
```

quantile



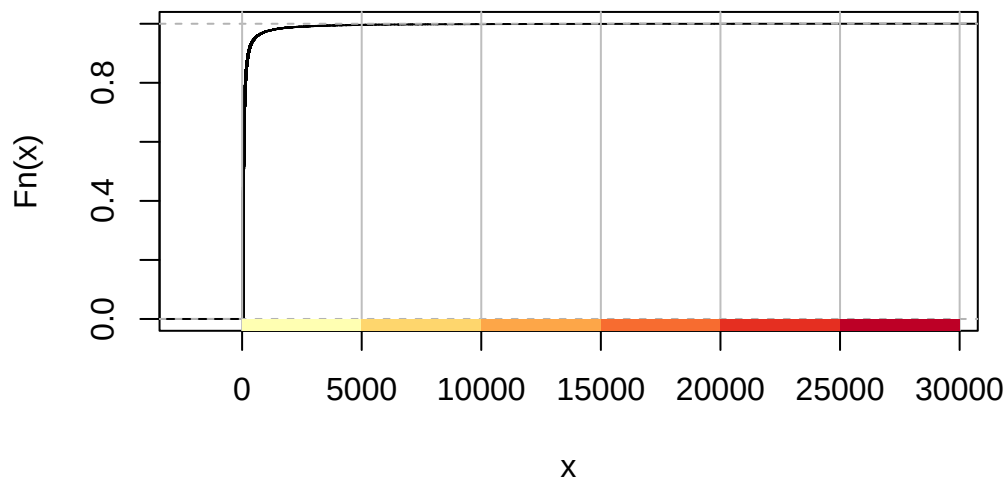
```
[1] -7249.988   163.246  7576.480 14989.714 22402.949 29816.183
```

sd



```
[1]      0    5000 10000 15000 20000 25000 30000
```

pretty

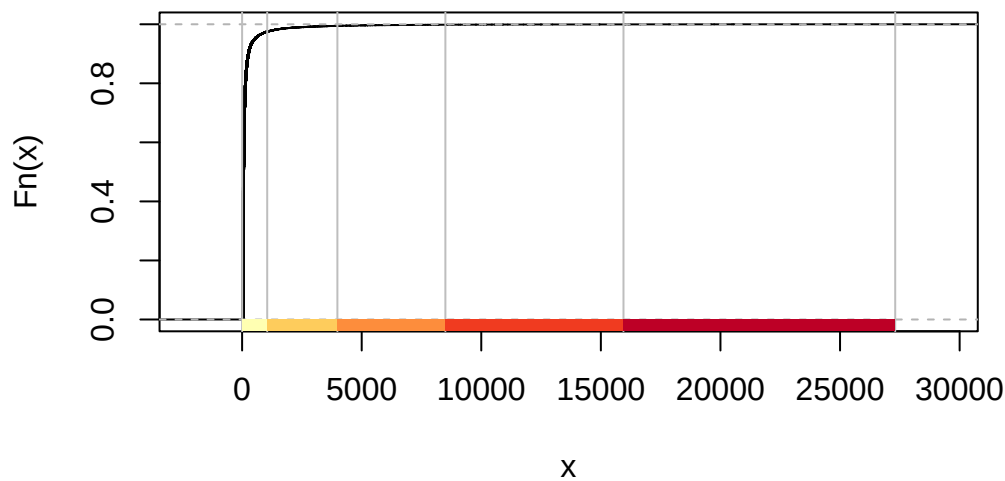


Warning in classIntervals(communes_fm\$densite, style = method, n = nb_classes):
N is large, and some styles will run very slowly; sampling imposed

Use "fisher" instead of "jenks" for larger data sets

```
[1] 0.000 1050.253 3984.769 8499.079 15944.915 27306.612
```

jenks



```
# A partir des informations obtenues, on peut définir nos propres intervalles.  
quantile(communes_fm$densite, probs = seq(0,1,0.1))
```

0%	10%	20%	30%	40%	50%
0.00000	9.76980	15.34789	21.55828	29.47814	40.33675
60%	70%	80%	90%	100%	
55.04587	78.09993	121.06509	244.57128	27306.61157	

```
summary(communes_fm$densite)
```

Min.	1st Qu.	Median	Mean	3rd Qu.	Max.
0.00	18.30	40.34	163.25	95.72	27306.61

```
# 40 = médiane
```

```
# 162 = moyenne
```

```
# On reprend certaines bornes de Jenks - en fusionnant les derniers intervalles
```

```
# Un exemple de découpage manuel avec 7 classes
denspop_man_brks7 <- c(0,40,162,500,1000,4000,8000,27200)
# Un exemple de découpage manuel avec 5 classes
denspop_man_brks5 <- c(0,40,162,1000,8000,27200)
```

- d. Finalement, on décide de discrétiser notre variable avec les bornes suivantes : $[0;40[$, $[40;162[$, $[162;1000[$, $[1000;8000[$ et $[8000;27200[$. Ajouter la variable discrétisée dans le fond communal. Vous utiliserez la fonction `cut()`. Vous ferez attention à l'inclusion des bornes inférieures et à l'exclusion des bornes supérieures.

```
popcomfm_sf <- communes_fm %>%
  mutate(
    densite_c = cut(
      densite,
      breaks = denspop_man_brks5,
      include.lowest = TRUE,
      right = FALSE,
      ordered_result = TRUE
    )
  )
```

- e. Analyser la distribution de cette variable. Représenter la variable discrétisée sur une carte, en créant préalablement une nouvelle palette de couleurs ayant le bon nombre de classes.

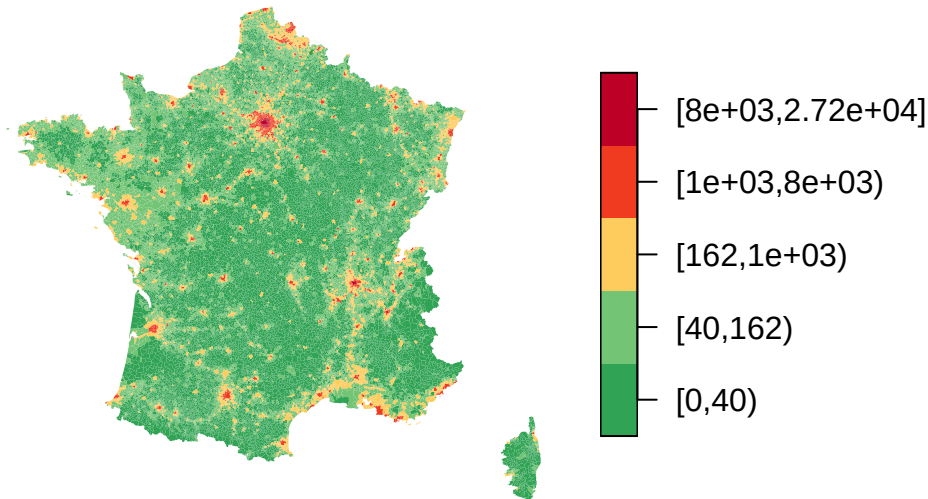
```
table(popcomfm_sf$densite_c)
```

[0,40)	[40,162)	[162,1e+03)	[1e+03,8e+03)
17327	12248	4371	827

[8e+03,2.72e+04]
62

```
pal2 <- c(
  RColorBrewer::brewer.pal(
    n=5,
    name="Greens"
  )[4:3],
  RColorBrewer::brewer.pal(
    n=5,
    name="YlOrRd"
  )[c(2,4:5)]
)
plot(
  popcomfm_sf["densite_c"],
  pal=pal2,
  border = FALSE,
  main = "Densité de population",
)
```

Densité de population



Exercice 2

Représenter sous forme de carte le taux de pauvreté par département. Vous utiliserez le package `mapsf`. Vous trouverez de la documentation sur ce package [ici](#). Vous utiliserez le fond “`dep_francemetro_2021.gpkg`” ainsi que le fichier “`Taux_pauvrete_2018.xlsx`”. Pour l’import de ce fichier, vous pouvez utiliser la fonction `openxlsx::read.xlsx()`.

1. Dans un premier temps, vous pourrez essayer de faire 3 cartes basiques : une en découpant la variables d’intérêt selon la méthode de Fisher (`breaks="fisher"`), une autre avec des classes de même amplitude (`"equal"`) et enfin selon la méthode des quantiles (`"quantile"`).

```
# Import des donnees
dep_francemetro_2021 <- st_read("fonds/dep_francemetro_2021.gpkg")

Reading layer `dep_francemetro_2021' from data source
  `/home/onyxia/work/2A_stat_spatiale/2025-2026/TP3/fonds/dep_francemetro_2021.gpkg'
  using driver `GPKG'
Simple feature collection with 96 features and 4 fields
Geometry type: MULTIPOLYGON
Dimension:      XY
Bounding box:   xmin: 99225.97 ymin: 6049647 xmax: 1242375 ymax: 7110480
Projected CRS:  RGF93 v1 / Lambert-93

tx_pauvrete <- read.xlsx("data/Taux_pauvrete_2018.xlsx")

# Import du fond de la mer (optionnel)
mer <- st_read("fonds/merf_2021.gpkg")

Reading layer `merf_2021' from data source
  `/home/onyxia/work/2A_stat_spatiale/2025-2026/TP3/fonds/merf_2021.gpkg'
  using driver `GPKG'
Simple feature collection with 4 features and 2 fields
Geometry type: MULTIPOLYGON
Dimension:      XY
Bounding box:   xmin: 2740.751 ymin: 5938801 xmax: 1344509 ymax: 7213433
Projected CRS:  RGF93 v1 / Lambert-93

# Résumé des jeux de données
str(dep_francemetro_2021)

Classes 'sf' and 'data.frame':  96 obs. of  5 variables:
 $ code   : chr  "01" "02" "03" "04" ...
 $ libelle: chr  "Ain" "Aisne" "Allier" "Alpes-de-Haute-Provence" ...
```



```

$ reg      : chr  "84" "32" "84" "93" ...
$ surf     : int   5785 7411 7380 6995 5690 4294 5567 5246 4911 6028 ...
$ geom     :sfc_MULTIPOLYGON of length 96; first list element: List of 1
  ..$ :List of 1
  .. ..$ : num [1:369, 1:2] 921435 920637 919201 918528 917850 ...
  ..- attr(*, "class")= chr [1:3] "XY" "MULTIPOLYGON" "sfg"
- attr(*, "sf_column")= chr "geom"
- attr(*, "agr")= Factor w/ 3 levels "constant","aggregate",...: NA NA NA NA
  ..- attr(*, "names")= chr [1:4] "code" "libelle" "reg" "surf"

```

```
str(tx_pauvrete)
```

```

'data.frame': 101 obs. of 3 variables:
 $ Code      : chr  "01" "02" "03" "04" ...
 $ Dept      : chr  "Ain" "Aisne" "Allier" "Alpes-de-Haute-Provence" ...
 $ Tx_pauvrete: num  10.3 18.4 15.5 16.8 13.9 15.8 14.4 18.9 18 16.2 ...

```

```

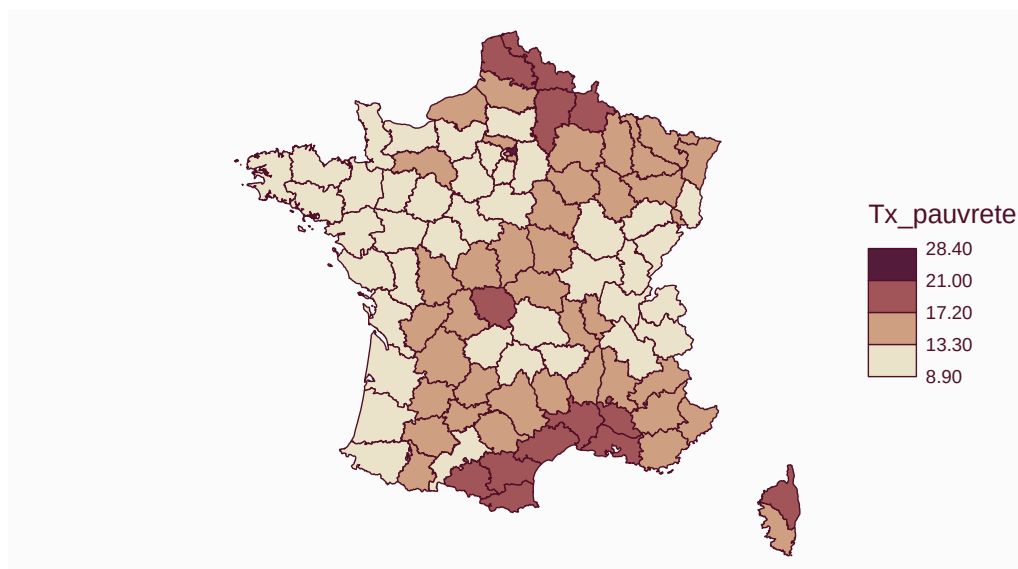
# Jointure pour rajouter le taux de pauvreté à notre fond départemental
dep_francemetro_2021_pauv<-dep_francemetro_2021 %>%
  left_join(tx_pauvrete %>% select(-Dept),
            by=c("code"="Code"))

```

```

# Représentation du taux de pauvreté avec la package mapsf
# La variable est de type ratio, il s'agit donc d'une carte choroplèthe.
# On peut déjà avoir une belle carte de manière très simple :
# Methode de Fisher
mf_map(x = dep_francemetro_2021_pauv,
       var = "Tx_pauvrete",
       type = "choro",
       nbreaks = 4,
       breaks= "jenks"
)

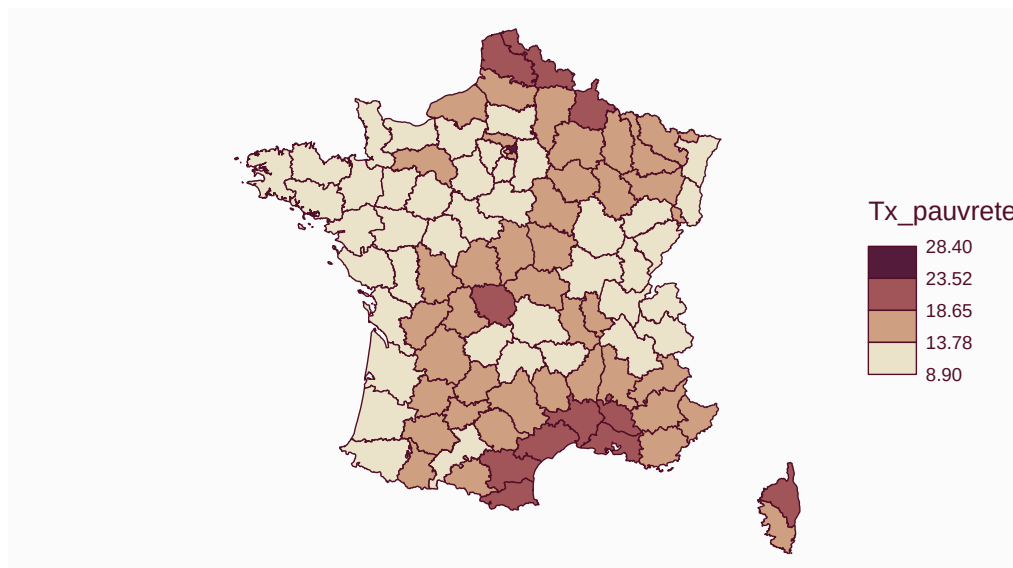
```



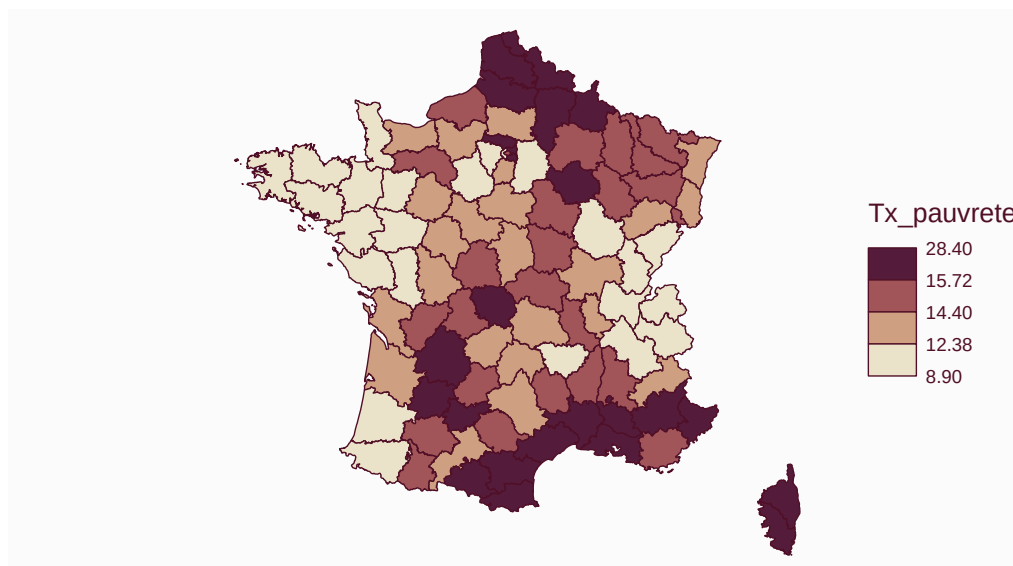
```

# Methode des classes de même amplitude
mf_map(x = dep_francemetro_2021_pauv,
       var = "Tx_pauvrete",
       type = "choro",
       nbreaks = 4,
       breaks= "equal"
)

```



```
# Methode des quantiles
mf_map(x = dep_francemetro_2021_pauv,
       var = "Tx_pauvrete",
       type = "choro",
       nbreaks = 4,
       breaks= "quantile"
)
```



2. Dans un 2e temps, vous ferez un découpages manuel avec les seuils suivants : 0, 13, 17, 25, $\max(\text{Tx_pauvrete})$. La carte contiendra également un zoom sur Paris et sa petite couronne (départements 75, 92, 93, 94).

```
# Réalisation d'une carte de quasi-diffusion, l'occasion de regarder les différentes options.
# Creation d'un vecteur de codes couleurs (découpage de la couleur bleu "Mint")
# On applique la fonction rev pour inverser l'ordre des codes :
# on souhaite avoir du plus clair au plus foncé
couleur <- rev(mf_get_pal(4,"Mint"))

mf_map(x = dep_francemetro_2021_pauv,
       # Variable à représenter
```

```

    var = "Tx_pauvrete",
    # Carte choroplethe
    type = "choro",
    # Seuils de découpage de notre variable
    breaks= c(0,13,17,25,max(dep_francemetro_2021_pauv$Tx_pauvrete)),
    pal = couleur,
    # leg_val_rnd = 0
    # On ne fait pas apparaitre la legende, on utilisera la fonction
    # mf_legend() pour personnaliser les libelles
    leg_pos = NA
)

# Creation d'un encadre pour notre carte (ouverture)
mf_inset_on(x = dep_francemetro_2021_pauv , pos = "topright",
           cex = .2)

# Pour centrer/zoomer sur notre encadre sur notre territoire
# (sinon Paris et sa couronne restent très petits)
mf_init(dep_francemetro_2021_pauv %>%
        filter(code %in% c("75","92","93","94")))

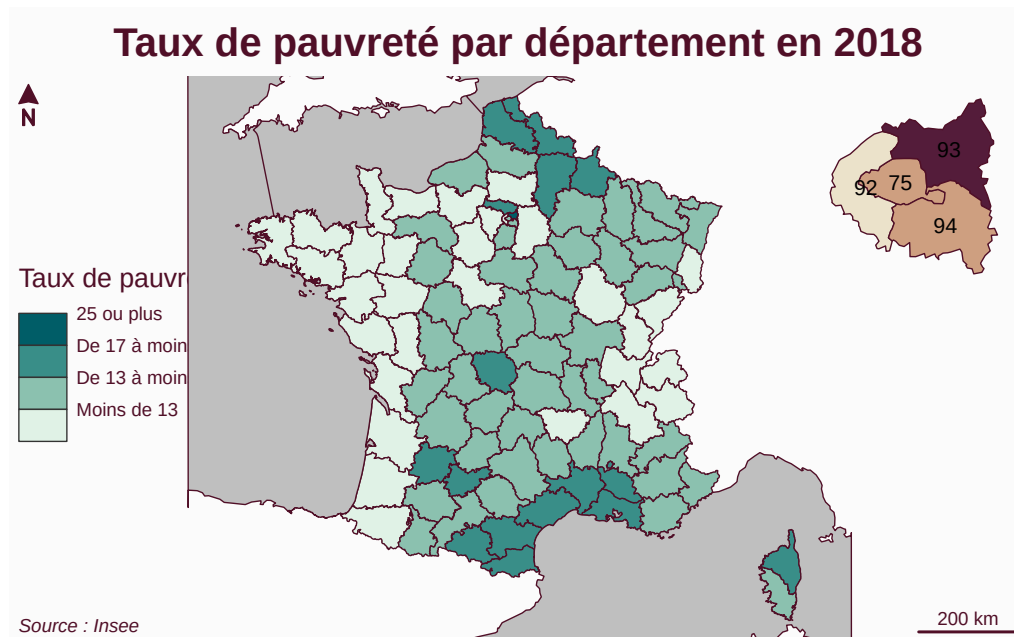
# On recrée la carte choroplethe sur Paris et sa couronne
mf_map(dep_francemetro_2021_pauv %>%
        filter(code %in% c("75","92","93","94")),
        var = "Tx_pauvrete",
        type = "choro",
        breaks= c(0,13,17,25,max(dep_francemetro_2021_pauv$Tx_pauvrete)),
        # Ne pas faire apparaître la legende de l'encadre
        leg_pos = NA,
        # Ne pas oublier le add=TRUE pour superposer l'encadre à notre carte
        add = TRUE)

# Rajout des codes départements
mf_label(dep_francemetro_2021_pauv %>%
        filter(code %in% c("75","92","93","94")),
        var = "code",
        col = "black")

# Fin de l'encadre (fermeture)
mf_inset_off()
# Rajout d'une legende
mf_legend(
  type="choro",
  title = "Taux de pauvreté",
  # Creation de labels personnalisés
  # On est obligé de mettre une modalite fictive (ici "") pour avoir une tranche
  val=c("", "Moins de 13", "De 13 à moins de 17", "De 17 à moins de 25", "25 ou plus"),
  pal=couleur,
  pos = "left"
)

# Si l'on veut rajouter la mer
mf_map(mer, add=TRUE)
mf_layout(title = "Taux de pauvreté par département en 2018",
          credits = "Source : Insee")

```



3. Sauvegarder le jeu de données ayant servi à faire la carte. Pour cela, vous utilisez la fonction `sf::st_write()`. Votre sauvegarde se fera sous le format `gpkg` (geopackage) sous le nom `"dept_tx_pauvrete_2018.gpkg"`.

```
st_write(dep_francemetro_2021_pauv, "dept_tx_pauvrete_2018.gpkg", delete_layer = T)
```

```
Deleting layer `dept_tx_pauvrete_2018' using driver `GPKG'
Writing layer `dept_tx_pauvrete_2018' to data source
`dept_tx_pauvrete_2018.gpkg' using driver `GPKG'
Writing 96 features with 5 fields and geometry type Multi Polygon.
```

Exercice 3

Dans la poursuite de la découverte de `mapsf`, réaliser une carte choroplèthe (sur une variable de densité) avec ronds proportionnels (sur une variable de population). Pour cela, vous utiliserez le **fond des régions de France métropolitaine** sur lequel vous calculerez une variable de densité. Pour la variable de population, vous disposez notamment du fichier **"pop_region_2019.xlsx"** présent dans le dossier **data**. La carte se fera avec l'option `type="prop_choro"` de la fonction `mf_map()`.

```
# Import du fond regional
region <- st_read("fonds/reg_francemetro_2021.gpkg")
```

```
Reading layer `reg_francemetro_2021' from data source
`/home/onyxia/work/2A_stat_spatiale/2025-2026/TP3/fonds/reg_francemetro_2021.gpkg'
using driver `GPKG'
Simple feature collection with 13 features and 3 fields
Geometry type: MULTIPOLYGON
Dimension: XY
Bounding box: xmin: 99225.97 ymin: 6049647 xmax: 1242375 ymax: 7110480
Projected CRS: RGF93 v1 / Lambert-93
```

```
# Import des donnees de population
pop_reg <- openxlsx::read.xlsx("data/pop_region_2019.xlsx")
```

```
# Import du fond de la mer (optionnel)
mer <- st_read("fonds/merf_2021.gpkg")
```

```
Reading layer `merf_2021' from data source
`/home/onyxia/work/2A_stat_spatiale/2025-2026/TP3/fonds/merf_2021.gpkg'
using driver `GPKG'
Simple feature collection with 4 features and 2 fields
Geometry type: MULTIPOLYGON
Dimension: XY
```

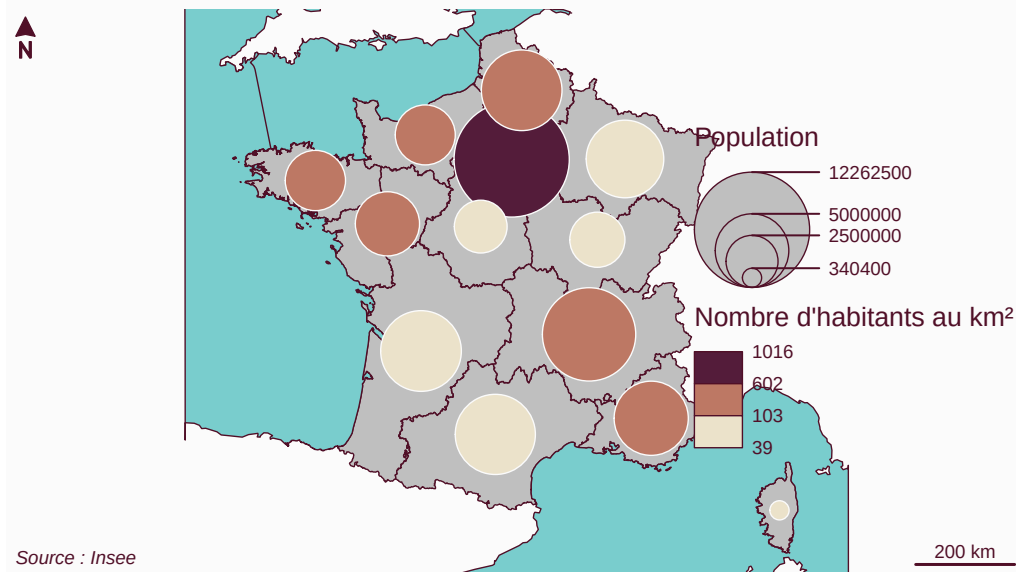
Bounding box: xmin: 2740.751 ymin: 5938801 xmax: 1344509 ymax: 7213433

Projected CRS: RGF93 v1 / Lambert-93

```
# On fait maintenant la jointure avec notre fond
region_pop<-region %>%
  left_join(pop_reg, by=c("code"="reg")) %>%
  mutate(densite=pop/surf)

# Creation de la carte
# Représentation du fond de la carte
mf_map(region_pop)
mf_map(mer, add=TRUE, col = "darkslategray3")
# Superposition des ronds proportionnels
mf_map(region_pop,
  var = c("pop", "densite"),
  nbreaks = 3,
  breaks="fisher",
  # Pour les options, il faut renseigner un vecteur à 2 éléments :
  # 1 pour la representation, 1 pour les ronds prop
  leg_val_rnd = c(-2,0),
  leg_title = c("Population", "Nombre d'habitants au km²"),
  type = "prop_choro")
mf_layout(title = "Densité et population des régions françaises en 2018",
  credits = "Source : Insee")
```

Densité et population des régions françaises en 2018



Exercice 4

1.a. Importer le **fond des communes de France métropolitaine** `commune_francemetro_2021.gpkg`.

```
comfm_sf <- st_read(  
  "fonds/commune_francemetro_2021.gpkg")  
str(comfm_sf)
```

- b. Importer le fichier `bpe20_sport_loisir_xy.csv` (séparateur de colonnes ; et séparateur des décimales .) et prenez connaissance des données. Il s'agit de la liste géolocalisée des équipements de sports et loisirs en France. **Une ligne est donc un équipement.** Pour la géolocalisation des équipements, le système de projection utilisé est le Lambert-93 (epsg=2154) pour la Métropole. Le séparateur de colonnes est le ;. Le fichier est issu du site de l'Insee : <https://www.insee.fr/fr/statistiques/3568638?sommaire=3568656>. Vous y trouverez la documentation nécessaire pour comprendre les données. Repérer notamment les variables renseignant les coordonnées géographiques des équipements, ainsi que la variable indiquant le type d'équipement.

```
# Utiliser l'argument n_max=1000 dans un premier temps pour vérifier le bon  
# chargement des données puis le retirer  
bpe20_sf <- read.csv(  
  "data/bpe20_sport_loisir_xy.csv",  
  sep = ";",  
  dec = "."  
)  
str(bpe20_sf)  
head(bpe20_sf)
```

2. Charger un fond OpenStreetMap centré sur la France métropolitaine. Pour cela, repérer les coordonnées (longitude/latitude) d'un point situé approximativement au centre de la France sur <https://www.openstreetmap.org/> (un clic-droit + afficher l'adresse). Ensuite utiliser le code suivant : `leaflet() %>% setView(lng = longitude, lat = latitude, zoom = 5) %>% addTiles()` où `longitude` et `latitude` correspondent aux coordonnées récupérées.

```
map <- leaflet() %>%  
  setView(lng = 2.6, lat = 47.2, zoom = 5) %>%  
  addTiles()  
map
```

3. Nous souhaitons positionner sur la carte l'emplacement des bowlings (TYPEQU == "F119") sur la carte.
- a. Récupérer de la base d'équipements les bowlings en France Métropolitaine (REG > 10) et les stocker dans une table à part. Retirer également les bowlings dont les coordonnées sont manquantes.

```
bowlings <- bpe20_sf %>%  
  filter(TYPEQU == "F119" & REG > 10 & !(is.na(LAMBERT_X)) & !(is.na(LAMBERT_Y)))
```

- b. Les cartes réalisées avec `leaflet` nécessitent d'utiliser des coordonnées dans le système de projection WGS84 - `epsg = 4326`. Transformer la base de bowlings en un objet `sf` avec la fonction `st_as_sf()` et en utilisant l'argument `coords =` pour préciser le nom des variables de coordonnées ainsi que l'argument `crs = 2154` (système de projection : RGF93). Ensuite, reprojecter le fond obtenu dans le système de projection adéquat avec la fonction `st_transform(crs = 4326)`.

```
bowlings_sf <- bowlings %>%  
  st_as_sf(coords = c("LAMBERT_X", "LAMBERT_Y"), crs = 2154) %>%  
  st_transform(crs = 4326)  
  
str(bowlings_sf)  
st_crs(bowlings_sf)
```

- c. Ajouter un marqueur localisant chacun des bowlings sur la carte interactive précédemment initialisée. Utiliser la fonction `addMarkers()` et l'argument `data =`.

```
map %>%
  addMarkers(
    data = bowlings_sf
  )
```

- d. Il est possible d'ajouter un **popup** qui affiche une information complémentaire quand on clique sur un marqueur. Reprendre le code précédent et ajouter l'argument **popup**=~DEPCOM pour afficher le code de la commune dans laquelle le bowling est installé.

```
map %>%
  addMarkers(
    data = bowlings_sf,
    popup = ~DEPCOM
  )
```

Une carte choroplèthe interactive

- 4.a. Initialiser un fond de carte interactif centré sur le département de votre choix. Adapter la démarche vue en 2.

```
# Les landes
map2 <- leaflet() %>%
  setView(lng=-0.5,lat=44,zoom=8) %>%
  addTiles()
map2
```

- b. Importer le **fichier base-cc-evol-struct-pop-2018_echantillon.csv**. Restreindre le tableau aux communes du département que vous aurez choisi et aux variables CODGEO, P18_POP (population communale) et P18_POP0014 (population de moins de 14 ans). Calculer la part des moins de 14 ans dans la population communale.

```
popcom_tb <- read.csv(
  "data/base-cc-evol-struct-pop-2018_echantillon.csv",
  sep = ",",
  dec = "."
) %>%
  filter(stringr::str_detect(CODGEO, "^40")) %>%
  select(CODGEO, P18_POP, P18_POP0014) %>%
  mutate(PART18_POP0014 = P18_POP0014/P18_POP*100)
```

- c. Fusionner le fond communal et le tableau que vous venez de constituer. Assurez-vous que le fond communal soit restreint aux communes du département choisi. Changer le système de projection pour qu'il soit de type WGS84.

```
popcomd40_sf <- comfm_sf %>%
  right_join(
    popcom_tb, by = c("code" = "CODGEO")
  ) %>%
  st_transform(crs = 4326)
```

- d. Ajouter à la carte interactive initialisée en a. les polygones du fond communal du département choisi. Utiliser la fonction **addPolygons** en utilisant les arguments **data** (le fond communal), **weight** (pour préciser l'épaisseur des bordures), **color** (couleur des bordures des polygones) et **fill=NA** (absence de couleur de remplissage).

```
map2 %>%
  addPolygons(
    data = popcomd40_sf,
    weight = 0.5, #width of the borders
    color = "purple",
    fill = NA
  )
```

- e. On souhaite ajouter une analyse thématique en représentant la part de moins de 14 ans dans chaque commune.

Pour cela, il faut préalablement discrétiser la variable étudiée. On utilisera une discrétisation par la méthode des quantiles avec la fonction `colorQuantile`. Ses arguments sont :

- `palette` = : nom d'une palette, par exemple "YlOrRd". Une liste de palettes disponibles peut s'obtenir avec `RColorBrewer::display.brewer.all()`.
- `domain` = : variable numérique à discrétiser.
- `n` = : nombre de classes

Créer une palette avec la fonction `colorQuantile()` en utilisant le modèle suivant :

```
pal <- colorQuantile(
  "Blues",
  domain = ma_table$var_a_discretiser,
  n = 5
)
```

```
pal <- colorQuantile(
  "Blues",
  domain = popcmd40_sf$PART18_POP0014,
  n = 5
)
```

- f. Ajouter l'analyse thématique à la carte interactive en ajoutant l'argument `fillColor` dans la fonction `addPolygons` utilisée en d. Retirer l'argument `fill` et ajouter l'argument `fillOpacity` pour gérer l'opacité de l'analyse thématique (1=opaque, 0=transparent).

```
map_choro <- map2 %>%
  addPolygons(
    data = popcmd40_sf,
    weight = 0.5, #width of the borders
    color = "purple",
    fillOpacity = 0.5,
    fillColor = ~pal(PART18_POP0014)
  )
map_choro
```

- g. Ajouter la légende de l'analyse avec la fonction `addLegend()`.

```
map_choro_leg <- map_choro %>%
  addLegend(
    pal = pal,
    values = popcmd40_sf$PART18_POP0014
  )
map_choro_leg
```

- h. On souhaite ajouter un popup à chaque commune affichant :

- le nom de la commune (variable `libelle`);
- la population de la commune (variable `P18_POP`);
- la part de la population de moins de 14 ans.

Pour cela, créer une variable `contenu_popup` dans le fond communal qui concatène les différentes informations ci-dessus. Pour ceux qui connaissent le html, on peut utiliser des balises html pour la mise en forme du popup sur le modèle suivant :

```
ma_table$contenu_popup <- paste0(
  "<b>", ma_table$libelle, "</b>",
  "<br>",
  "Population: ",
  format(ma_table$population, big.mark = " "),
  "<br>",
  "Part moins de 15 ans: ",
  round(ma_table$part_moins_15ans, 1),
  "% "
)
```



```

popcmd40_sf$contenu_popup <- paste0(
  "<b>", popcmd40_sf$libelle, "</b>",
  "<br>",
  "Population: ",
  format(popcmd40_sf$P18_POP, big.mark = " "),
  "<br>",
  "Part moins de 15 ans: ",
  round(popcmd40_sf$PART18_POP0014, 1),
  "%"
)

```

Ajouter ensuite à la fonction `addPolygons()` l'argument `popup = ~contenu_pop`.

```

map_choro_popup <- map2 %>%
  addPolygons(
    data = popcmd40_sf,
    weight = 0.5, #width of the borders
    color = "purple",
    fillOpacity = 0.5,
    fillColor = ~pal(PART18_POP0014),
    popup = ~contenu_popup
  ) %>%
  addLegend(
    pal = pal,
    values = popcmd40_sf$PART18_POP0014
  )
map_choro_popup

```

- i. On souhaite enfin ajouter l'emplacement des bassins de natation. Construire une base des équipements de natation (`TYPEQU == "F101"`) situés dans le département choisi. Transformer ce dataframe en un objet `sf` comme en 3.b

```

natation_d40 <- bpesl_tb %>%
  filter(TYPEQU == "F101" & DEP == "40" & !(is.na(LAMBERT_X)) & !(is.na(LAMBERT_Y)))

natation_sf <- natation_d40 %>%
  st_as_sf(coords = c("LAMBERT_X", "LAMBERT_Y"), crs = 2154) %>%
  st_transform(crs = 4326)
str(natation_sf)
st_crs(natation_sf)

```

Enfin, ajouter les emplacements des bassins de natation sur la carte interactive communale. Inspirez-vous de 4.c

```

map_choro_popup %>%
  addMarkers(
    data = natation_sf,
    label = ~as.character(DEPCOM)
  )

```

- j. La dernière étape consiste à ajouter un contrôle des couches à faire apparaître.

Reprendre l'ensemble du code de fabrication de la carte interactive pour constituer un seul bloc de code.

Ajouter l'argument `group=` avec un nom identifiant la couche dans chaque fonction correspondant à une nouvelle couche.

Ajouter une dernière instruction à l'enchaînement en utilisant la fonction `addLayersControl()` et l'argument `overlayGroups =` en y listant les différents groupes de couches.

```

leaflet() %>%
  setView(lng=-0.5, lat=44, zoom=8) %>%
  addTiles() %>%
  addPolygons(
    data = popcmd40_sf,
    weight = 0.5, #width of the borders

```

```

    color = "purple",
    fill = NA,
    group = "limites communales"
  ) %>%
  addPolygons(
    data = popcomd40_sf,
    weight = 0.5, #width of the borders
    color = NA,
    fillOpacity = 0.5,
    fillColor = ~pal(PART18_POP0014),
    popup = ~contenu_popup,
    group = "Analyse thématique"
  ) %>%
  addLegend(
    pal = pal,
    values = popcomd40_sf$PART18_POP0014
  ) %>%
  addMarkers(
    data = natation_sf,
    label = ~as.character(DEPCOM),
    group = "Bassins de natation"
  ) %>%
  addLayersControl(
    overlayGroups = c("limites communales", "Analyse thématique", "Bassins de natation")
  )

```

Bonus : Faire une carte interactive plus simplement avec le package mapview

L'idée est de réaliser une carte équivalente à celle réalisée dans la question précédente.

```
library(mapview)
```

On crée nos classes à partir d'une discrétisation de la part de moins de 14 ans par la méthode des quantiles (ici des quintiles).

```

qt_part <- quantile(
  popcomd40_sf$PART18_POP0014, probs = seq(0,1,.2)
)
qt_part_arr <- c(
  floor(qt_part[1]*10)/10,
  round(qt_part[2:5], 1),
  ceiling(qt_part[6]*10)/10
)

```

On réalise ensuite la carte avec **mapview**, en construisant dans un premier temps la carte choroplèthe des parts de moins de 14 ans par communes (1er appel à la fonction **mapview()**), et en y ajoutant (comme dans la logique **ggplot**) les autres couches nécessaires (ici : les bassins de natations). On peut se permettre de facilement représenter le type de bassin (couvert ou non couvert) en plus de leur localisation.

```

mapview(
  # fond de polygone
  popcomd40_sf,
  # variable représentée (possible d'en mettre plusieurs)
  z = c("PART18_POP0014"),
  # Les bornes des intervalles pour des classes manuelle
  at = qt_part_arr,
  # transparence du remplissage des polygones
  alpha.regions = 0.35,
  # Nom de la couche
  layer.name = "Moins de 14 ans (en %)",
  #Info affichée quand on survole le polygone => ici = libellé de la commune
  label = "libelle",
  # Infos affichées quand on clique sur un polygone
  popup = leafpop::popupTable(popcomd40_sf, z = c("code","libelle","P18_POP", "PART18_POP0014"))
) +

```

```

mapview(
  # sf composé de points
  natation_sf %>%
    # on en profite pour transformer la variable COUVERT en facteur
    mutate(
      COUVERT = factor(
        COUVERT,
        levels = 0:1,
        labels = c("Non couverte", "Couverte")
      )
    ),
  z = "COUVERT",
  # les couleurs des points selon la catégorie de la variable COUVERT
  col.regions = c("steelblue", "coral"),
  # transparences
  alpha = 0.8,
  alpha.regions = 0.7,
  # Nom de la couche
  layer.name = "Piscines",
  # Popup: affichage quand on clique sur un point
  popup = leafpop::popupTable(natation_sf, z = "NB_AIREJEU")
)

```

COMMENTAIRES/INDICATIONS

Exercice 1

Une représentation réalisée directement sur une variable continue peut être pertinente quand le nombre d'observations est très grand et quand la distribution est suffisamment équilibrée. Ce n'est souvent pas le cas.

La discrétisation

Il est alors préférable de discrétiser la variable continue, c'est-à-dire de répartir les observations dans un certain nombre de classes.

Pour cela, il faut pouvoir déterminer une méthode et un nombre de classes. Les méthodes les plus utilisées sont les suivantes :

- **les quantiles** : chaque classe est composée du même nombre d'observations ;
- **les seuils dits “naturels”** (k-means, jenks, fisher) : les classes sont construites pour optimiser le rapprochement des valeurs similaires et ainsi obtenir des classes les plus différentes entre elles. Quand l'algorithme des k-means est généralisable à un espace de dimension n , l'algorithme de Jenks est optimisé pour les variables à une dimension qui nous occupe ici.
- **les écarts-types** : la construction des classes est réalisée à partir de la moyenne de la distribution avec une longueur d'un écart-type. Cette méthode est adaptée aux variables suivant approximativement une distribution normale.
- **les seuils manuels** : Les seuils sont choisis manuellement. Cette méthode peut être utilisée pour affiner les classes obtenues avec une méthode automatique.

Exercices 2 et 3

Vous pouvez aussi vous reporter au lien [suivant](#).

Exercice 4

Pas mal de documentation sur leaflet avec R :

- [le blog thinkR](#) (blog sur R avec souvent de très bons tutoriels)
- [site de Laurent Rouvière](#) (tres complet sur R)
- [github Rstudio](#) présentant leaflet